付完款只是开始

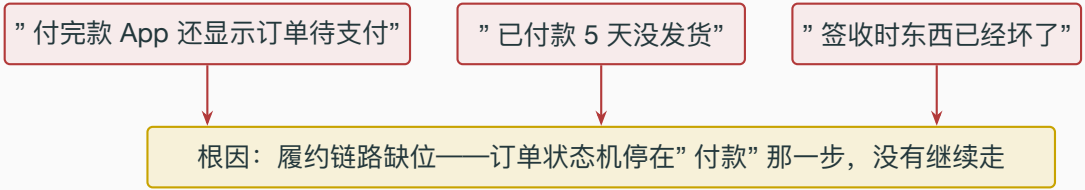
电商履约链路全景与 gomall 路线图 (业务侧 deck)

gomall · 业务侧 deck

今日大纲

业务困局与完整状态机

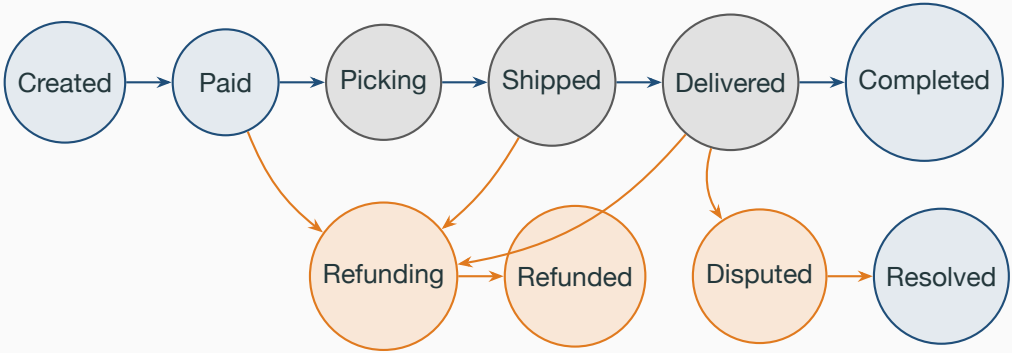
客服后台每天收到的三类工单



- C1 是支付到订单同步**慢/丢**：典型对账问题，已付款事件没及时落到订单上。
- C2 是发货 SLA 缺失：商家什么时候必须发货，平台没承诺、没监控。
- C3 是签收和售后链路缺失：东西坏了走哪条退货流程，没定义。

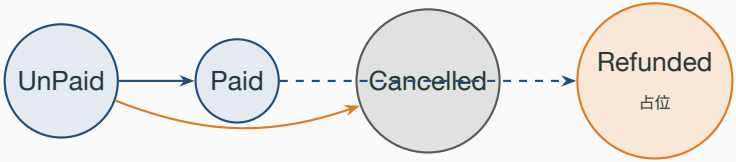
consts/order.go:11–15 当前状态枚举只有 *UnPaid / Paid / Cancelled / Refunded*

电商订单状态机：完整版



主链 6 态 `Created` → `Paid` → `Picking` → `Shipped` → `Delivered` → `Completed`；分支两条：`Refunding` → `Refunded`（金额回退）和 `Disputed` → `Resolved`（售后争议）。京东/淘宝/天猫公开订单 *API* 状态枚举综合归纳

gomall 现状与差距对照



完整链路	gomall	缺失原因	客诉表现
Created/Paid	✓	—	—
Picking	**缺**	仓储未接	” 下单后没动静”
Shipped	**缺**	物流单未建	”5 天没发货”
Delivered	**缺**	物流回调未接	” 看不到包裹到哪”
Completed	**缺**	收货确认未做	” 钱啥时放给商家”
Refunding/Refunded	枚举有 / 无写入	退款 Saga 未做	” 退款卡住”

缺 **4–5 个核心状态**。每一个缺位对应至少一类客诉。*consts/order.go:4–7, 11–15*

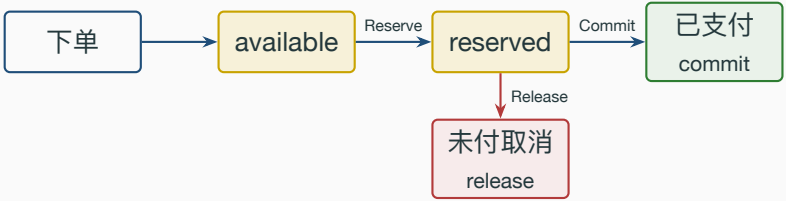
履约五段拆解与现有积木

把”付款到收货”切成五段

阶段	业务动作	gomall 现状
仓储	库存预扣 → 实扣 → 拣货单	预扣有，拣货单 **缺**
出库	拣货 → 复核 → 打包 → 出库单	**全缺**
物流	物流单 → 揽收 → 中转 → 派送	**全缺**
签收	收件人确认 → 订单推进 Completed	**缺**
售后	退换货 → 退款 Saga	**缺**

”缺”的根因不是技术做不到——MVP 阶段先聚焦**交易闭环**，履约依赖第三方仓 / 物流。自营 SKU 占比上来后五段都要补。*repository/cache/inventory.go:11–14* 库存双桶模型 *(available/reserved)*

阶段 1 仓储：唯一已经站住的一段



- Redis 双桶模型 (`available` / `reserved`) 在下单 / 支付 / 关单三条路径上落实原子语义。
- **缺的不是”扣库存”，是”出库的下一步动作”**：拣货单（从哪个货架挑哪几件）。
- 自营仓场景下，拣货单是 WMS 的事；gomall MVP 没接 WMS，所以这一步只是**数字游戏**——DB 里 `product.Num` 扣了，没有”物理上挑出来”这一步。

repository/cache/inventory.go:32–66 Reserve/Commit/Release Lua；service/payment.go:128–143 DB 真扣

阶段 2–5 的客诉成本

缺位环节	客诉成本	为什么 gomall 暂缺
出库单	”已付款 5 天没发货”	平台未承诺出库 SLA，商家拖单无监控
拣货 / 复核	错发 / 漏发 / 多发	自营仓未建，依赖商家自管
物流建单	”我的订单怎么没单号”	物流商接口未对接
揽收 / 中转	”包裹卡在路上没动”	物流商 webhook 未接
签收确认	”钱啥时放给商家”	无确认收货链路
售后退换货	”退款卡住” / ”差评”	Refunded 枚举无写入路径

共性：**都依**

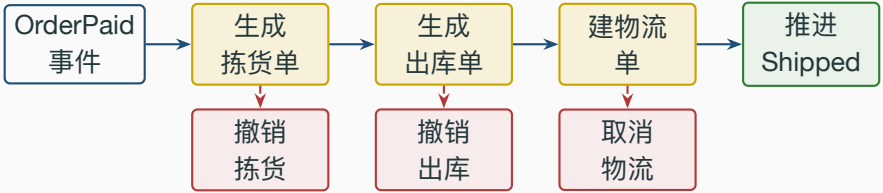
赖第三方 webhook 或客服后台。一旦接入，回调可靠性、签名、重放、乱序就是新的工程难题。*routes/router.go* 无 */logistics/** 路由；*service/payment.go:104–126* 付款即放款

gomall 已有积木一览

积木	来自	履约场景里能干什么
Outbox 模式	deck 04 / service/outbox	跨服务事件投递 at-least-once
Saga 协同	deck 04 / 事件驱动	退款 / 出库失败的多步回滚
RMQ 延迟队列	deck 15 / order_delay	超时未发货 / 超时未签收自动推进
Cron 兜底	deck 15 / order_task	RMQ 挂时的最后一道闸
幂等 SQL	deck 15 / WHERE 状态	物流回调重放安全
库存双桶	deck 11 / cache/inventory	退款时把库存还回 available

履约链路新增代码量大，但**底层一致性机制不用重新造**：每一段都能从这六块积木里挑两三块拼起来。*service/outbox/publisher.go:39–62* 现有 *Publisher*；*repository/rabbitmq/order_delay.go:24–56* 延迟拓扑

Outbox + Saga 在履约链路里怎么用



- 每一步都写一条新 outbox 事件 (`order.picking` / `order.shipped` / ...)。
- 任一步失败 → 走对应**补偿事件**（reverse arrow），最终回到一个一致状态。
- 协同式 Saga 没有中心 coordinator，靠 RMQ topic 串起来——与 deck 04 现状一致。

service/events/events.go:1–33 现有事件；docs/slides/04-outbox-saga.tex §Saga

RMQ 延迟队列的多场景复用

场景	延迟	触发动作
30 分钟未付款	30 min	CancelUnpaidOrder（已落地，deck 15）
72 小时商家未发货	72 h	自动通知 + 客服介入工单
15 天未签收	15 d	自动确认收货 + 放款给商家
30 天未发起售后	30 d	关闭售后入口

RMQ TTL +

死信交换的拓扑是**可复用**的：每个延迟场景只需要新加一对 `*.delay.queue` / `*.dead.queue`，PublishDelayed 函数签名一模一样。*repository/rabbitmq/order_delay.go:24–56* 拓扑模式可复刻

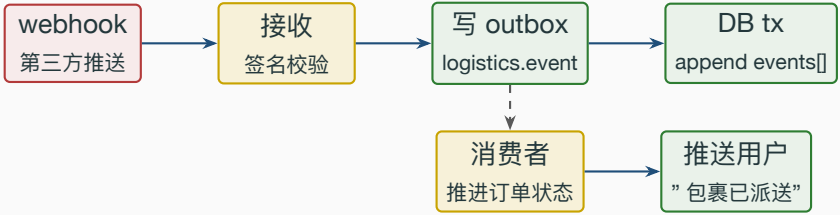
物流单 + 退款 Saga 路线图

物流单的最小业务字段

字段	业务含义
`logistics_no`	物流商单号（顺丰 SF... / 京东 JD...）
`carrier`	承运方编码（SF / JD / YTO / ...）
`from_zone`	发货仓 / 起站
`to_zone`	收货地址 / 终站
`status`	揽收 / 中转 / 派送 / 签收 / 异常
`events[]`	节点时间线（时间、地点、动作）
`sla_promised_at`	平台承诺送达时间

events[] 是 append-only 节点流（类似消息队列），status 是 events 末态的物化视图。这种” 事件流 + 物化态” 模型与 gomall 现有的 outbox 设计天然契合。未来 *repository/db/model/logistics.go* 设计草案

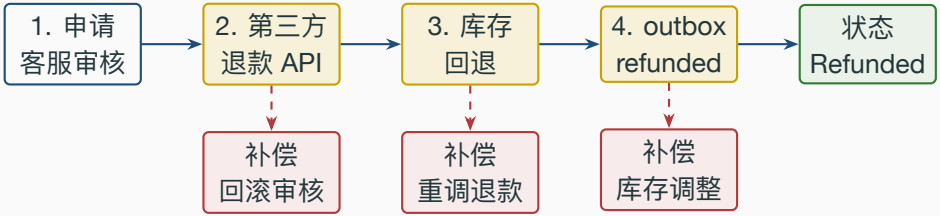
物流商 webhook → outbox 链路



- webhook 不可靠（重试 / 乱序 / 丢失）——用（`logistics_no, event_seq`）作幂等键。
- 落 outbox 而不是直接改订单：万一订单服务挂了，事件不会丢。
- 消费者负责把**物流节点**翻译为**订单状态推进**（揽收 → Shipped、签收 → Delivered）。

service/outbox/publisher.go:65–86 现有 *publisher* 模式可复用

退款发起后必须做的四件事



- 任一步失败 → 触发对应补偿事件 + **客服工单介入**（金钱链路不允许全自动回滚）。
- 第三方退款 API 是**慢依赖**：套 deck 05 的 CircuitBreaker，避免 goroutine 堆积。
- 调用必须**带商户单号**做幂等键——网关重试不能扣两次。

deck 04 outbox-saga 模式；middleware/circuitbreaker.go:44–128 复用三态熔断

为什么退款不能像下单那样原子

下单链路	退款链路	差异
单进程 + 单 DB tx	跨第三方支付	没有跨边界事务
失败回滚 = DROP 一切	钱已打出去无法回滚	只能补偿调用
错了客服可以手动重发	重发 = 多付一次资损	强幂等是底线
30min 内必关单 SLA	客服审核 1–3 天	长周期 Saga

退款是**典型的**

Saga 场景：跨服务、长事务、有补偿、有人工。Outbox 保证事件 at-least-once，每一步的 handler 强幂等保证 at-most-once 效果——加起来才是恰好一次。

> 退款 Saga 的最高优先级**不是**" 做得快"，是" **钱不丢、不重**"。

service/payment.go:128–143 DB tx 内下单原子 *vs* 跨第三方退款不可能原子

业务边界与各角色视角

自营 vs 第三方履约

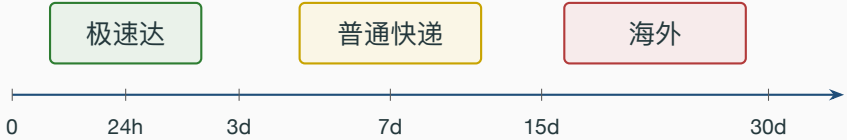
维度	自营	第三方
仓	平台自建 / 租赁	商家自管
拣货系统	平台 WMS	商家自管
物流	平台自配 / 优选承运	商家自选
SLA	平台兜底 + 赔付	商家承担
gomall	暂**无**	当前所有 SKU 默认走第三方

gomall MVP 把**所有**

SKU 当作第三方履约处理——平台只管下单 / 付款 / 关单。这是合理的工程取舍，但带来一个问题：商家”漏发 / 拖单”时平台没数据可看，只能等客诉。

repository/db/model/order.go:8–18 BossID 字段隐含”商家自营”模型

SLA 分档与售后时窗



品类	售后时窗	业务含义
普通商品	签收 + 7 天	行规”七天无理由”
3C / 大件	签收 + 15 天	给装机 / 调试时间
食品 / 鲜花	不支持	易腐，签收后无法退回
定制 / 虚拟	不支持	个性化或一次性消费

售后时窗 = **放款给商家的延迟**。普通商品 放款 = 签收 +7d（覆盖退款窗）；食品 放款 = 签收当天。现状：gomall 在付款时直接给商家加钱，平台实际承担退款资损，长期不可持续。*service/payment.go:104–126* 当前付款即放款 *boss* 余额

各角色关心的指标

角色	关心	看哪个
用户	啥时候到	物流节点时间线 + 预计送达
商家	客户什么时候确认收货	出库率 / 派送率 / 签收率
客服	客诉处理时长	退款审核 SLA / 工单 backlog
平台	物流商履约	各承运方签收率 / 异常率分布
财务	何时给商家结算	平台在途资金 / 待放款

同一份履约数据 (物流 events 表) 喂**五个不同的看板**。每个角色都是数据下游，不应该让他们各自查 DB ——消费 `logistics.event` 写五份物化视图。未来：消费 *logistics.event* 写五份物化视图

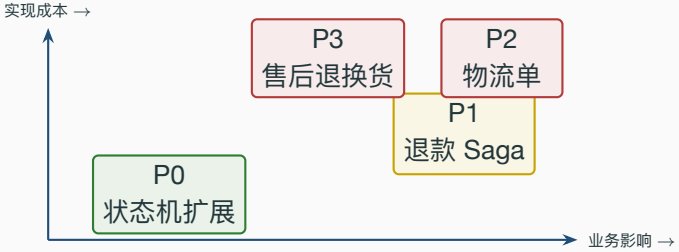
压测数据告诉我们什么

接口	实测 RPS	p95
/orders/create (幂等)	50,319	2.33ms
/orders/list (游标)	58,406	5.0ms
/ping 基线	64,254	3.51ms

- 交易链路已经压到 50K+ RPS（pre-fulfillment 段绰绰有余）。
- 履约接口建立后，新增压力主要在**后台异步链路**：物流 webhook 入站、消费者吃 events、推送出站。
- **HTTP 链路不会成为瓶颈**：RMQ 未启动场景下交易 HTTP 仍 0 错（见 REPORT），履约消费者挂掉同样不影响下单。

stressTest/REPORT.md 各链路压测表；orders/create 50,319 RPS / p95 2.33ms

优先级路线图：业务影响 × 实现成本



- **P0**：状态机扩到 6–7 态。代码量小，杜绝”枚举占位但无写入”的脏状态。
- **P1**：退款 Saga。资金链路兜底，避免客服手工改 DB。
- **P2**：物流单接入。对接物流商 webhook + 签名 + 重放防护。
- **P3**：售后退换货。除代码外还要客服后台 UI。

优先级估算 = 客诉占比 × 单位修复成本

P0 状态扩展的最小变更面

- `consts/order.go` 新增 `Picking` / `Shipped` / `Delivered` / `Completed` 四个状态。
- `service/payment.go:146` 不直接跳到 `PendingShipping`，改为发 `order.paid` 事件，由消费者推进。
- 新增 `service/fulfillment_consumer.go`：吃 `order.paid` → 写 `Picking`；吃 `logistics.shipped` → 写 `Shipped`；以此类推。
- 关键约束：**每一步推进都必须用 WHERE 状态 = 上一态**（沿用 deck 15 的 CloseOrderWithCheck 模式），保证幂等 + 不可回退。

> P0 不引入新表 / 新依赖，只补状态枚举 + 一个消费者——一周可上线。

consts/order.go:4–7；service/payment.go:146；repository/db/dao/order.go:185–196 幂等 SQL 模式

用路线图把三类客诉对回去

客诉	缺失环节	路线图阶段
”付完款仍显示待支付”	支付事件未及时落订单	P0 状态扩展 + 消费者
”已付款 5 天没发货”	出库 SLA 未承诺	P0 状态机 + RMQ 72h 告警
”签收时东西已经坏了”	售后链路未建	P3 售后退换货

三类客诉 → 三个阶段 → **P0 一上线就能消化 2/3**。P3 上线之前，售后只能继续走客服手工——这是**业务承诺要诚实告诉用户**的事。客诉分类来自客服后台分类口径（行业惯例）

履约不是技术决策，是业务承诺

- 状态机扩到 6–7 态：是**对用户**的承诺——” 我会一直让你知道订单走到哪一步”。
- 出库 / 签收 SLA：是**对商家**的承诺——” 拖单会有告警 / 自动通知客服介入”。
- 退款 Saga：是**对财务**的承诺——” 钱不会丢、不会重、所有路径有审计轨迹”。
- 物流单 + webhook：是**对平台自己**的承诺——” 我对履约的可视性，不依赖商家诚信”。

付完款只是开始——履约链路才是平台**真正交付的产品**。

业务 Q&A（上）

Q1：为什么 MVP 不一开始就做完整状态机？

A：MVP 阶段 SKU 全部走第三方履约，平台对” 商家什么时候发货” 没有数据来源——把 `Shipped / Delivered` 状态画上去也只是写死的字符串。等接入物流 webhook 后状态才有真实数据驱动，那时候再扩才不是占位。

Q2：退款失败钱去哪了？

A：钱在第三方支付商那里。第三方退款 API 失败分两种：网络失败（钱没动）和” 已受理但状态未回”（钱可能在路上）。前者直接重试，后者必须靠商户单号查询接口对账。Saga 第二步的补偿不是” 再调一次”，是” 先查再决定”。

Q3：物流商 webhook 不可靠怎么办？

A：webhook 重试 / 乱序 / 丢失全部存在。对策三件套：(1) 用 `(logistics_no, event_seq)` 强幂等键；(2) 落 outbox 而不是直接改订单；(3) 定时主动查询物流商 API 作兜底，类似 deck 15 的”RMQ + Cron 双保险”。

业务 Q&A（下）

Q4：海外履约链路和国内有什么差异？

A：差三件事——(1) 时窗：海外签收 15–30 天，退款时窗放宽到 30 天；(2) 跨境清关阶段不计入 SLA（行业惯例），订单状态停在 InCustoms；(3) 退款币种汇率锁定问题，原币种退还是按当日汇率退要财务先定。

Q5：大件 / 小件 / 虚拟商品的链路差异？

A：大件要约送货时间 + 上门安装；小件标准快递；虚拟商品（充值卡 / 软件激活码）**跳过出库 + 物流**，付款成功直接 Completed。这就是为什么状态机推进逻辑要**按 SKU 类型**选择路径，不能写死。

Q6：物流 SLA 怎么订才合理？

A：先看物流商自己的承诺（顺丰次日达、京东当日达），再砍 10% 给自己留余量——例如承运商承诺 24h，平台承诺 24h × 1.1 = 26h 作为告警阈值。物流商真的不达标，平台要么换承运、要么对用户赔付。

代码位置一览

订单状态枚举（待扩） *consts/order.go:4–7, 11–15*

订单字段（待加 logistics_no） *repository/db/model/order.go:8–18*

下单 + 预扣 + outbox *service/order.go:40–102*

付款 + 真扣 + 直接放款给商家 *service/payment.go:35–200*

关单链路（已落地，参 deck 15） *service/order_cancel.go:19–63*

库存双桶 Lua *repository/cache/inventory.go:32–66*

现有 outbox publisher *service/outbox/publisher.go:39–86*

现有事件 payload *service/events/events.go:1–52*

RMQ 延迟拓扑（可复用） *repository/rabbitmq/order_delay.go:24–56*

路线图新增：fulfillment_consumer.go *service/fulfillment_consumer.go （P0 待新建）*

路线图新增：物流单模型 *repository/db/model/logistics.go （P2 待新建）*

路线图新增：退款 Saga 入口 *service/refund_saga.go （P1 待新建）*